**МИНОБРНАУКИ РОССИИ**
**САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ**
**ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ**
**«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)**

# ОТЧЁТ

**по лабораторной работе № 1**

**по дисциплине «Программирование»**

**Тема: Типы данных и их внутреннее представление в памяти**

Студентка гр. 6372 _____ Олешко М.П.

Санкт-Петербург

2026

## 1. Цель работы

Целью лабораторной работы является изучение особенностей внутреннего представления целых и вещественных чисел в памяти компьютера, а также освоение методов анализа и изменения отдельных битов.

## 2. Содержание работы

В ходе работы были рассмотрены способы кодирования знаковых целых чисел, формат IEEE 754 для типов float и double, специальные значения, побитовые логические операции и получение объектного представления чисел с помощью std::memcpy.

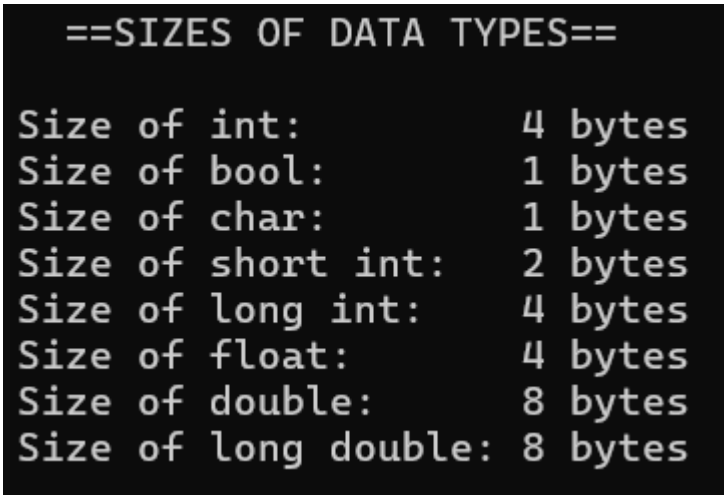
В практической части разработана программа на языке C++, которая выводит размеры основных типов данных, отображает двоичное представление целых и вещественных чисел, позволяет изменять отдельные биты и выполняет индивидуальное задание — умножение на 2 с помощью битового сдвига.

## 3. Вывод размеров типов данных

Для определения объёма памяти, занимаемого различными типами данных, в программе реализована функция printTypeSizes(). Она использует оператор sizeof, который возвращает размер типа или объекта в байтах. С помощью этой функции выводятся размеры типов int, bool, char, short int, long int, float, double и long double.

Полученные значения позволяют сравнить объём памяти, необходимый для хранения разных типов данных. Например, вещественные типы float и double отличаются не только размером, но и точностью представления чисел. Следует учитывать, что размеры некоторых типов, в частности int, long int и long double, могут различаться в зависимости от платформы и используемой реализации компилятора.

Вывод:




**4. Представление целых чисел в памяти**

Целые числа хранятся в памяти в виде последовательности битов. Для беззнаковых типов все разряды используются для представления числового значения. Для знаковых типов необходимо также представить знак числа.

Основными способами представления знаковых целых чисел являются прямой, обратный и дополнительный коды. В прямом коде старший разряд обозначает знак, а остальные разряды содержат модуль числа. В обратном коде отрицательное число получают инвертированием всех битов положительного числа. В дополнительном коде после инвертирования битов прибавляют единицу.

Например, для восьмиразрядной записи числа −6 соответствующие варианты имеют вид: прямой код — 10000110; обратный код — 11111001; дополнительный код — 11111010.

В современных вычислительных системах знаковые целые числа обычно представляются в дополнительном коде. В программе для вывода двоичного представления типа int используется std::bitset<32>. Старший бит выводится отдельно, чтобы обозначить знаковый разряд. Для unsigned int реализован отдельный вывод без знакового бита.

Вывод:

```
  ==BINARY UNSIGNED IN:==

Enter an unsigned int number:
23

Number:       23
Sign bit:      absent (unsigned type)
Bits:          00000000 00000000 00000000 00010111

  ==BINARY INT==

Enter an int number:
-41

Number:       -41
Binary number: 1 1111111 11111111 11111111 11010111
The first bit is the sign bit.
```

## 5. Формат IEEE 754 для вещественных чисел

Для представления вещественных чисел типов float и double обычно используется стандарт IEEE 754. Представление состоит из трёх полей: знакового бита, экспоненты и мантиссы.

| Тип | Всего бит | Экспонента | Мантисса |
|---|---|---|---|
| float | 32 | 8 | 23 |
| double | 64 | 11 | 52 |

Знаковый бит определяет знак числа. Экспонента хранит порядок со смещением: 127 для float и 1023 для double. Мантисса содержит дробную часть значащих цифр числа.

Пример вывода:

```
  ==BINARY FLOAT==


Number:   3.14
Sign bit: 0
Exponent: 10000000
Mantissa: 1001000 11110101 11000011


Number:   -2.5
Sign bit: 1
Exponent: 10000000
Mantissa: 0100000 00000000 00000000
```

```
  ==BINARY DOUBLE==


Number:   3.14
Sign bit: 0
Exponent: 10000000000
Mantissa: 1001 00011110 10111000 01010001 11101011 10000101 00011111


Number:   -2.5
Sign bit: 1
Exponent: 10000000000
Mantissa: 0100 00000000 00000000 00000000 00000000 00000000 00000000
```

### 5.1. Нормализованные и денормализованные числа

Для нормализованных чисел поле экспоненты не является полностью нулевым или полностью состоящим из единиц. В значащей части подразумевается ведущая единица. Если поле экспоненты состоит из нулей, а мантисса не равна нулю, число является денормализованным. Такие значения позволяют представлять числа, близкие к нулю.

Пример вывода:

```
Number:   0
Sign bit: 0
Exponent: 00000000
Mantissa: 0000000 00000000 00000000


Number:   1.4013e-45
Sign bit: 0
Exponent: 00000000
Mantissa: 0000000 00000000 00000001


Number:   inf
Sign bit: 0
Exponent: 11111111
Mantissa: 0000000 00000000 00000000
```

```
Number:   0
Sign bit: 0
Exponent: 00000000000
Mantissa: 0000 00000000 00000000 00000000 00000000 00000000 00000000


Number:   4.94066e-324
Sign bit: 0
Exponent: 00000000000
Mantissa: 0000 00000000 00000000 00000000 00000000 00000000 00000001


Number:   inf
Sign bit: 0
Exponent: 11111111111
Mantissa: 0000 00000000 00000000 00000000 00000000 00000000 00000000
```

### 5.2. Специальные значения

Стандарт IEEE 754 предусматривает положительную и отрицательную бесконечность, NaN и два нуля — положительный и отрицательный. Бесконечность кодируется единичной экспонентой и нулевой мантиссой. NaN имеет единичную экспоненту и ненулевую мантиссу.

В программе для демонстрации используются значения 3.14, −2.5, 0.0, минимальное положительное денормализованное значение и бесконечность для типов float и double.

## 6. Получение объектного представления с помощью std::memcpy

Для анализа битового представления вещественного числа необходимо получить последовательность байтов, хранящуюся в памяти. Обычное числовое преобразование float в целый тип не подходит, поскольку оно преобразует значение, а не сохраняет его битовый образ.

В программе используется std::memcpy из заголовочного файла <cstring>. Для float байты копируются в переменную std::uint32_t, а для double — в std::uint64_t. Затем полученную последовательность можно анализировать с помощью std::bitset.

Для float выводятся знаковый бит 31, экспонента (биты 30–23) и мантисса (биты 22–0). Для double выводятся знаковый бит 63, экспонента (биты 62–52) и мантисса (биты 51–0). Размеры проверяются перед копированием.

**7. Операции изменения отдельных битов**

В программе реализована функция modifyBit, которая принимает исходное значение, номер бита и код операции. Для формирования маски единица сдвигается влево на номер выбранного разряда.

Для установки бита используется операция OR: value |= mask. Для сброса применяется AND с инвертированной маской: value &= ~mask. Для инверсии используется XOR: value ^= mask.

В программе операция 1 устанавливает бит, операция 2 сбрасывает его, а операция 3 инвертирует. Реализованы перегруженные функции для 32-разрядного и 64-разрядного беззнаковых целых типов. Проверка номера бита предотвращает обращение за пределы допустимой разрядности.

Демонстрация изменения битов предусмотрена для int, а также для битовых представлений float и double.

Примеры вывода:

```
  ==MANIPULATION WITH BITS==

Enter data type (1 - int, 2 - float, 3 - double): 1
Enter an int number: 25

Original Binary number: 00000000 00000000 00000000 00011001

Enter bit number (0-31): 23
Enter operation (1 - set, 2 - reset, 3 - invert): 1


Result: 00000000 10000000 00000000 00011001
```

```
  ==MANIPULATION WITH BITS==

Enter data type (1 - int, 2 - float, 3 - double): 2
Enter a float number: 24.5

Original Binary number: 01000001 11000100 00000000 00000000

Enter bit number (0-31): 2
Enter operation (1 - set, 2 - reset, 3 - invert): 3


Result: 01000001 11000100 00000000 00000100
```

```
  ==MANIPULATION WITH BITS==

Enter data type (1 - int, 2 - float, 3 - double): 3
Enter a double number: 12.45

Original Binary number: 01000000 00101000 11100110 01100110 01100110 01100110 01100110 01100110

Enter bit number (0-63): 43
Enter operation (1 - set, 2 - reset, 3 - invert): 3


Result: 01000000 00101000 11101110 01100110 01100110 01100110 01100110 01100110
```

**8. Индивидуальное задание: умножение на 2**

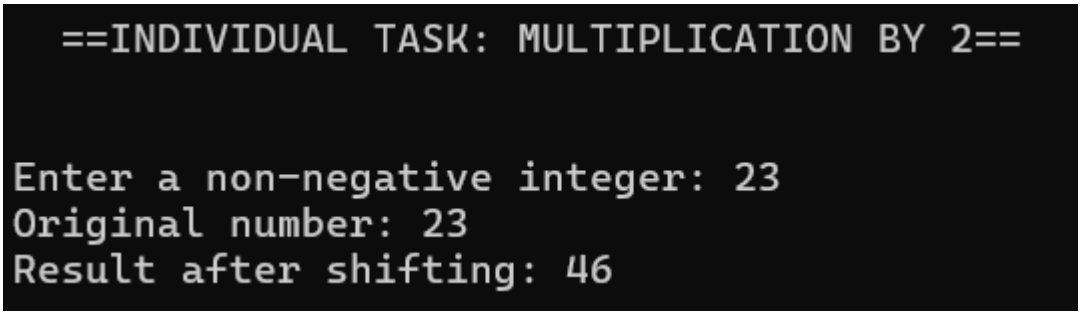
Индивидуальное задание состоит в реализации умножения целого числа на 2 с использованием битового сдвига. Для неотрицательного целого числа, если результат представим типом int, сдвиг влево на один разряд соответствует умножению на 2.

Например, двоичная запись числа 12 имеет вид 00001100. После сдвига влево на один разряд получается 00011000, что соответствует десятичному числу 24.

В программе вычисление выполняется выражением result = num << 1.

Отрицательные числа и значения, для которых результат не помещается в int, должны приводить к сообщению об ошибке.

Пример вывода:




**9. Структура программы. Рефакторинг**

После реализации программа была разделена на отдельные модули. Файл main.cpp содержит главную функцию и последовательный запуск этапов. Модуль type_info.h/type_info.cpp отвечает за вывод размеров типов данных. Модуль binary_output.h/binary_output.cpp содержит функции вывода двоичного представления. Модуль bit_operations.h/bit_operations.cpp реализует изменение битов. Модуль individual_task.h/individual_task.cpp содержит индивидуальное задание.

В заголовочных файлах используется директива #pragma once для предотвращения повторного включения заголовка. Для хранения битовых последовательностей фиксированной ширины применяются типы std::uint32_t и std::uint64_t из <cstdint>.

**10. Выводы**

В ходе лабораторной работы были изучены способы представления целых чисел и особенности формата IEEE 754 для вещественных типов float и double. Были рассмотрены знаковый бит, экспонента, мантисса, субнормальные числа и специальные значения.

С помощью std::memcpy было получено объектное представление вещественных чисел. Реализованы операции установки, сброса и инверсии отдельных битов, а также вывод двоичного представления целых и вещественных значений.

В индивидуальном задании реализовано умножение неотрицательного целого числа на 2 с помощью сдвига влево на один разряд. Рефакторинг позволил разделить программу на логические модули и упростить её сопровождение.